Kingdom of Saudi Arabia
Imam Abdulrahman Bin Faisal University
College Of Science and Humanities-Jubail
Computer Science Department

JavaScript Implementation Screenshots & Documentation

Group 4

#	Student Id	Student name
1	2230006809	Zahra Hussain Alabdulatif
2	2230004065	Razan Alqahtani
3	2230004865	Aljazi eddah alsaiari
4	2220001189	Zainab alkathim
5	2230000188	Fatima Musleh Alomar
6	2230000758	Maha Alharthi
7	2230007336	Rehaf Awad Alotaibi

Supervised by:

Ms. Raja Alyafer

Ms. Reem Alshammari

Web Development Project
2026

Aura Artifacts — E-Commerce Platform
Screenshots demonstrating the implementation of JavaScript within the project, including Web Services & API integrations

Table of Contents

1. Dark Mode Toggle 3
2. Collections Filter & Category System 5
3. Product Search — Live Results 7
4. Currency Converter — External API Integration 8
5. Bracelet Customization Builder 10
6. Product Detail — Image Gallery & Interactions 12
7. Error Handler — Toast Notifications 14
Web Services & APIs Integrated into the System 16
API-1: Currency Exchange Rate API 16
API-2: Google Maps Embed API 16

Web Services & APIs:

API-1: Currency Exchange Rate API (open.er-api.com)

API-2: Google Maps Embed API (maps.google.com)

1. Dark Mode Toggle

File: assets/js/dark-mode.js

This JavaScript file implements a theme switching feature that allows users to toggle between light and dark modes. The user's preference is saved in localStorage, ensuring it persists across page reloads and browser sessions.

How it works:

• A toggle button in the navbar switches between light (🌙) and dark (🌞) themes
• The script sets the data-theme attribute on the HTML element
• A separate CSS file (dark-mode.css, 509 lines) overrides all colors for dark mode
• The theme is applied before the page renders to prevent a 'flash' of the wrong theme
• localStorage stores the preference permanently

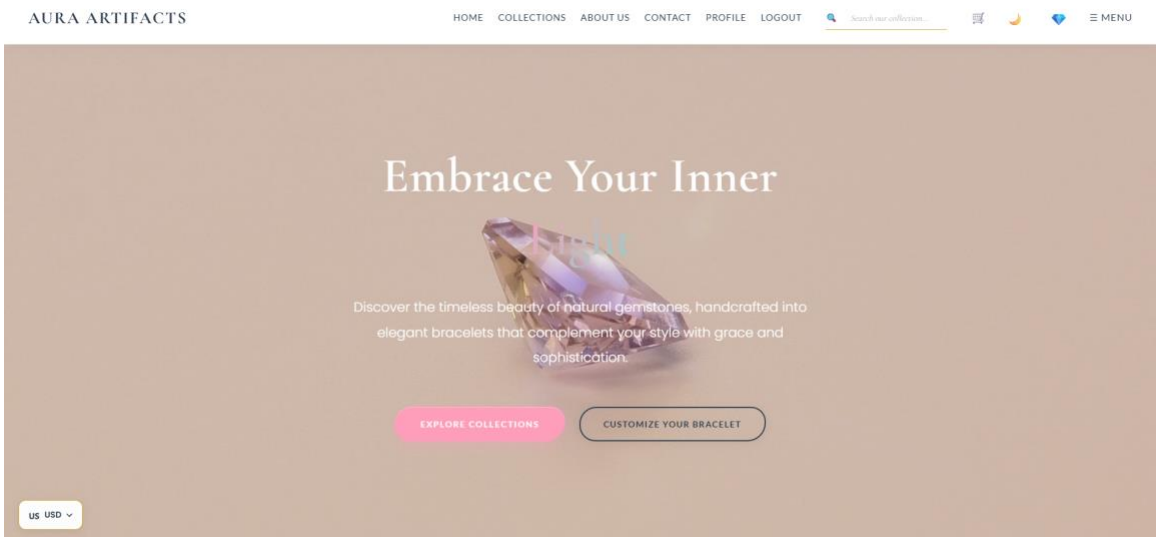


Figure: Homepage in Light Mode — default theme with light backgrounds and dark text

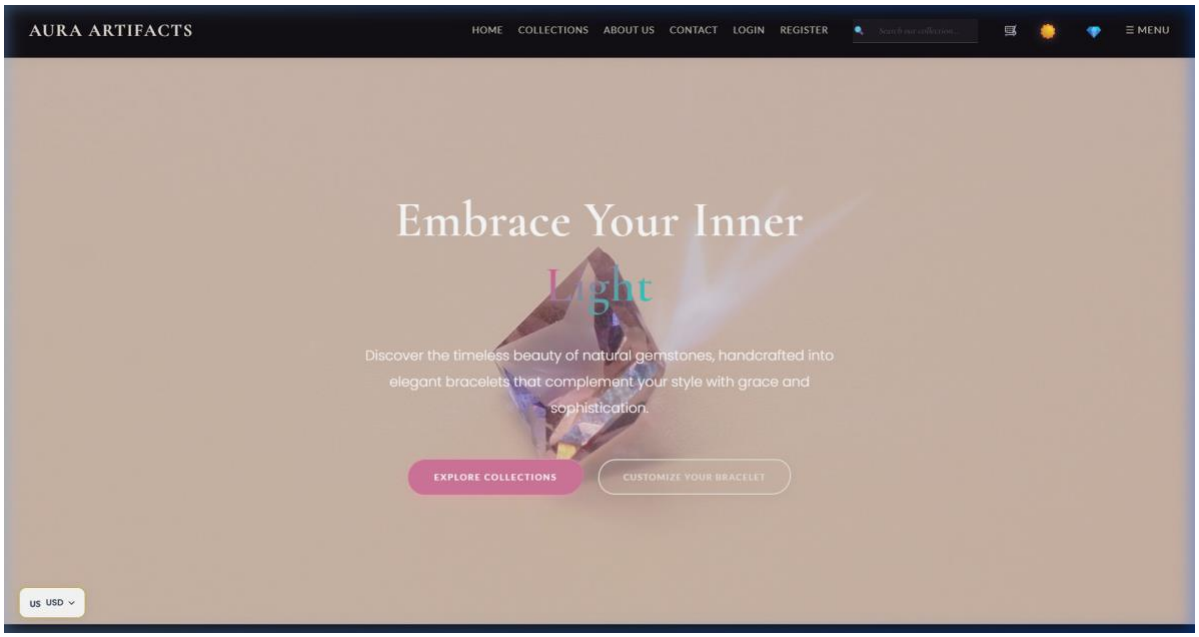


Figure: Homepage in Dark Mode — after clicking the toggle button, all colors switch to a dark palette

2. Collections Filter & Category System

File: assets/js/collections.js

This script implements a dynamic product filtering system on the Collections page. Users can filter products by category without reloading the page.

How it works:
• Filter buttons at the top: ALL, BRACELETS, RINGS, PENDANTS, EARRINGS, ANKLETS, SUNGLASSES
• Clicking a button filters products using CSS class matching
• Products that don't match the selected category are hidden with a fade animation
• The 'ALL' button shows all products
• Active button gets a highlighted style to indicate the current filter
• Skeleton loading animation shows while products load

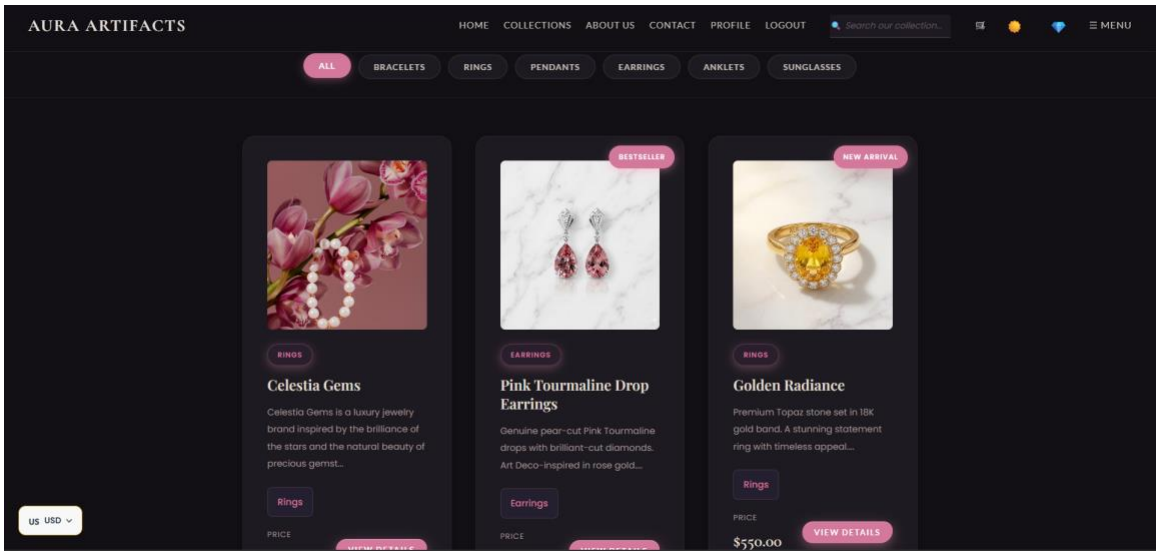


Figure: Collections page showing ALL products with filter buttons at the top (ALL is active)

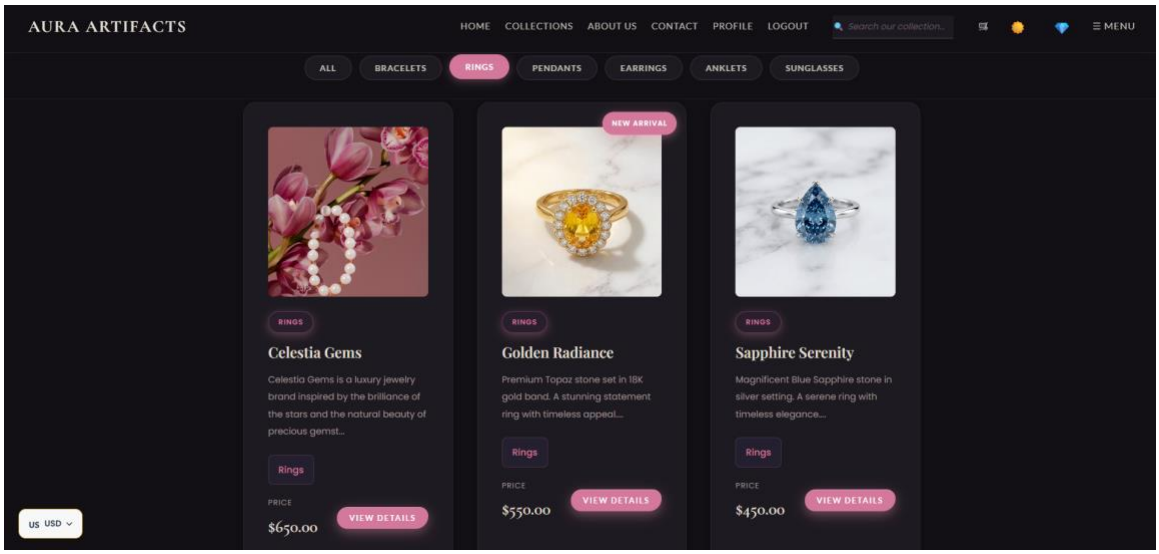


Figure: Collections page filtered to show only RINGS category — other products are hidden

3. Product Search — Live Results

File: assets/js/product-detail.js (searchProducts function)

This feature provides instant search functionality across the website. As the user types in the search bar, matching products appear in a dropdown.

How it works:
• An input field in the navbar triggers the search on each keystroke
• The searchProducts() function filters a product array by name and category
• Matching results appear in a dropdown below the search input
• Each result is a clickable link to the product detail page
• Clicking outside the dropdown closes it
• The search is case-insensitive and matches partial text

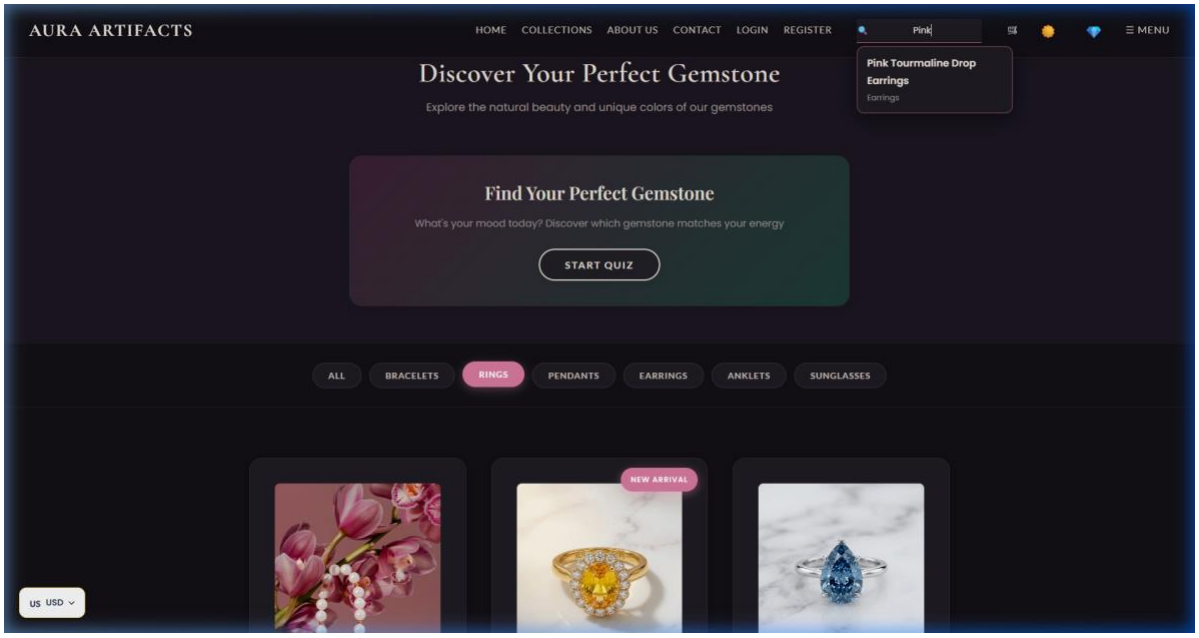


Figure: Search results dropdown showing products matching 'Pink' — results include product names and categories

4. Currency Converter — External API Integration

File: assets/js/currency-converter.js
API: https://open.er-api.com/v6/latest/USD

This is the primary external API integration in the project. It fetches live exchange rates and converts all product prices on the page to the user's selected currency.

How it works:
• A floating button at the bottom-left of every page shows the current currency
• Clicking it opens a dropdown with 150+ currencies, organized by popularity
• The dropdown includes a search bar to quickly find currencies
• Each currency shows its flag emoji, code, full name, and exchange rate
• Selecting a currency converts all prices on the page with a smooth animation
• Converted prices show the original USD value below for reference
• The selected currency is saved in localStorage for persistence
• Exchange rates are cached locally for 1 hour to minimize API calls
• If the API is unavailable, fallback rates are used for 10 popular currencies

API Details:
• Provider: ExchangeRate-API (open.er-api.com)
• Endpoint: GET https://open.er-api.com/v6/latest/USD
• Authentication: None required (free tier)
• Response: JSON with 150+ currency rates relative to USD
• Technology: JavaScript fetch() API with async/await

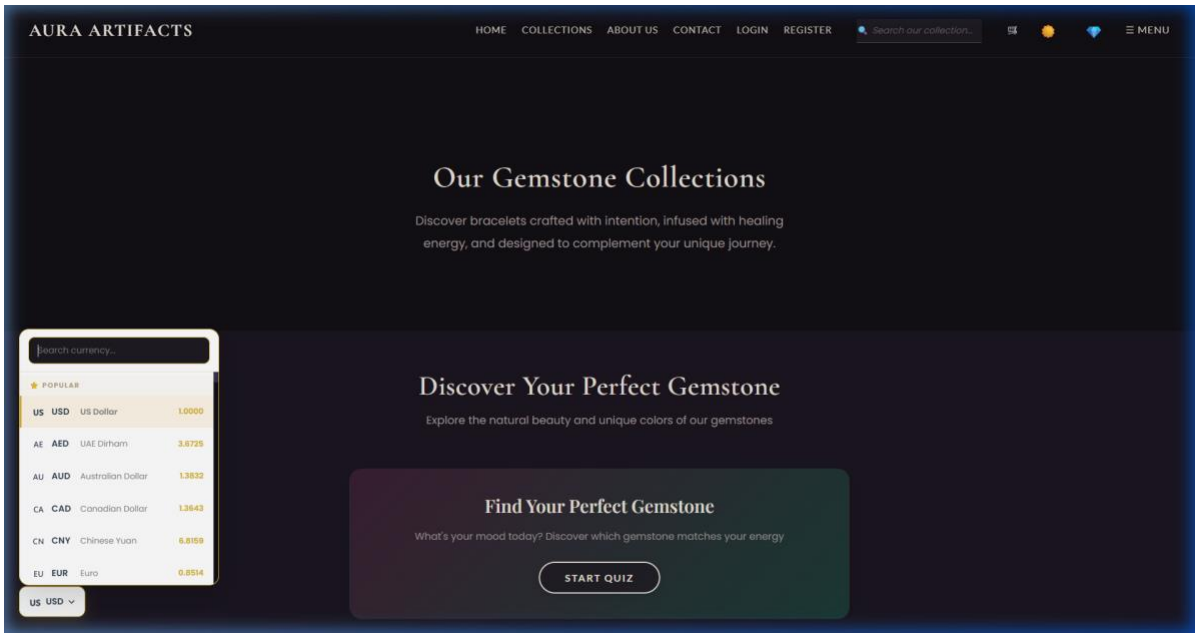


Figure: Currency dropdown open — showing popular currencies with flags, codes, names, and live exchange rates. Search bar at the top for filtering.

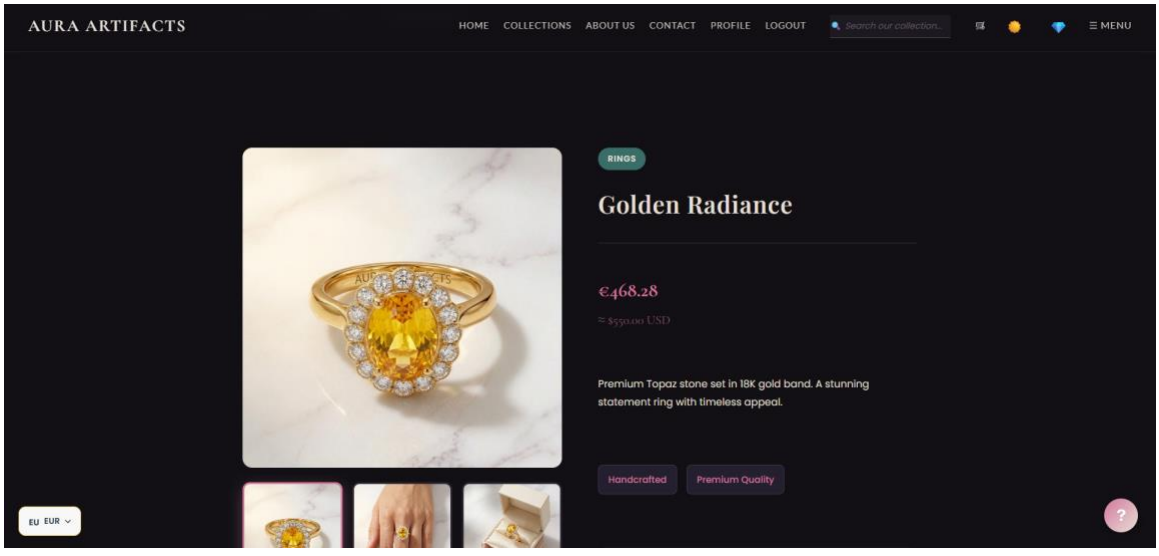


Figure: Prices converted to EUR (Euro) — each price shows the converted amount with the original USD value displayed below

5. Bracelet Customization Builder

File: assets/js/bracelet-customization.js

This is a unique interactive feature that allows users to design their own custom bracelets by selecting individual charms from 13 different collections.

How it works:
• 13 charm categories are dynamically generated from folder names
• Each category displays 5 charm options as clickable cards
• Clicking a charm adds it to the preview area and increases the total price
• Base price is $0, each charm adds $5
• Minimum 10 charms required to enable the 'Add to Cart' button
• Users can undo the last charm or clear all selections
• The preview area shows all selected charms in real-time
• Submitting sends the order via fetch() API as a JSON POST request
• The server saves a BraceletPreview record and adds it to the cart

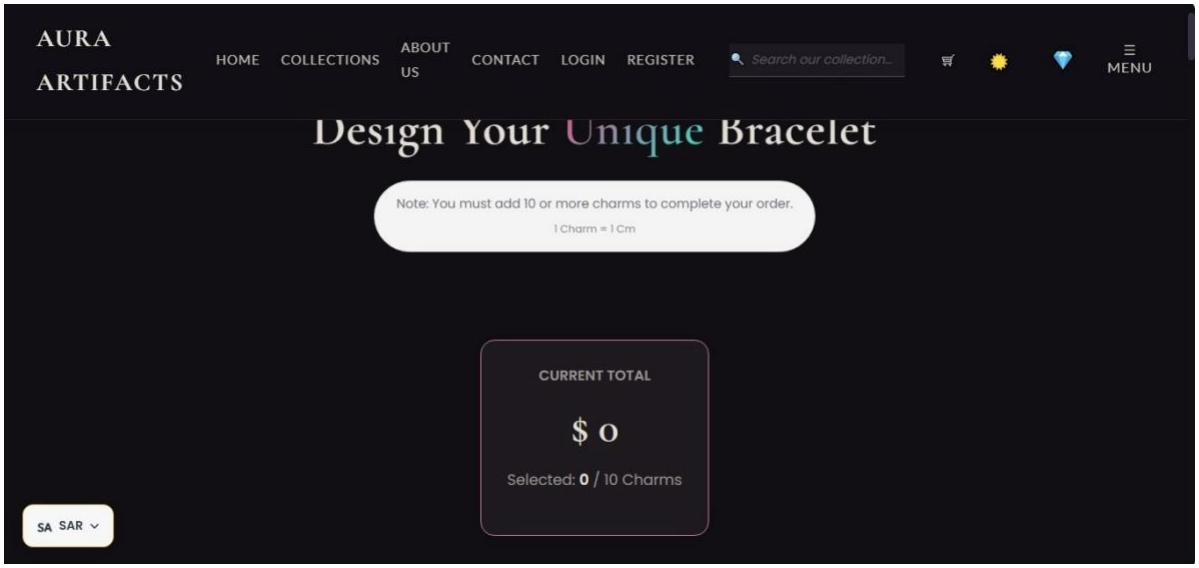


Figure: Bracelet Customization page — showing charm categories grid with clickable charm cards

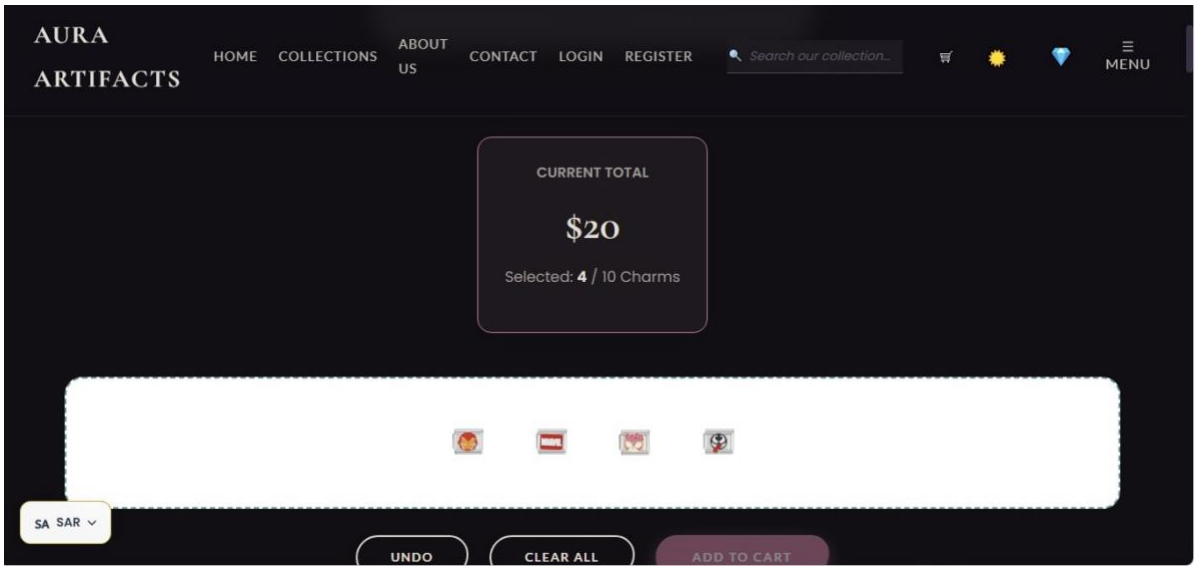


Figure: Charm selected and added to the bracelet preview area — total price updated dynamically

6. Product Detail — Image Gallery & Interactions

File: assets/js/product-detail.js

This script handles the interactive features on the product detail page, including the image gallery, quantity adjustment, and help popup.

How it works:

• Image Gallery: Up to 3 product images displayed as thumbnails
  - Clicking a thumbnail changes the main display image
  - Active thumbnail gets a highlighted border
• Quantity Selector: +/- buttons to adjust purchase quantity
• Help Popup: Opens a new browser window with product assistance
  - window.open('help', 'helpWindow', 'width=600,height=400')
• Recently Viewed: Tracks last 5 viewed products using cookies

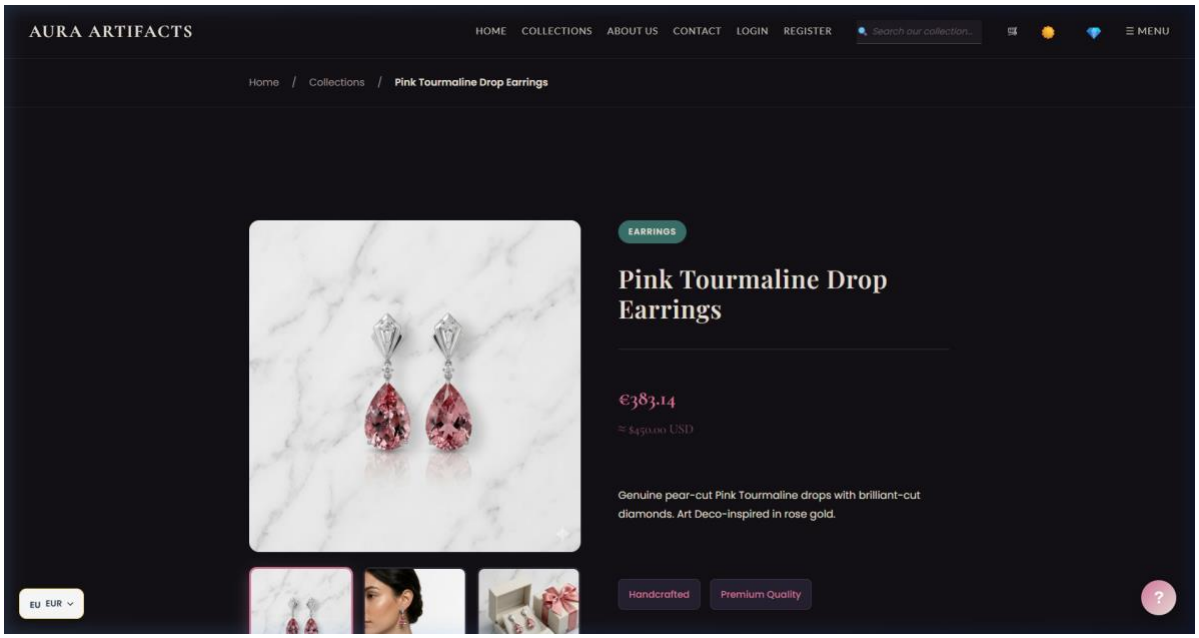


Figure: Product detail page showing the main image, thumbnails, price, and Add to Cart button

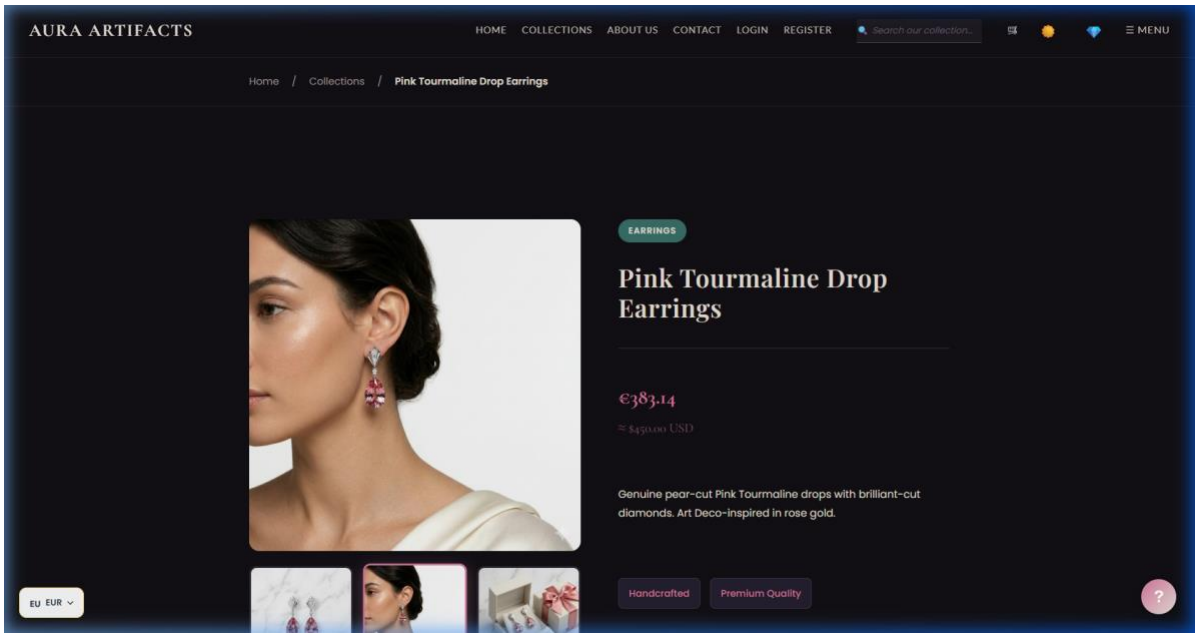


Figure: Image gallery — clicking a thumbnail changes the main product image displayed above

7. Error Handler — Toast Notifications

File: assets/js/error-handler.js

This script provides a modern toast notification system that replaces traditional browser alerts and page-level error messages.

How it works:
• showErrorToast(message, type) — displays a floating notification
• Types: 'success' (green), 'error' (red)
• Notifications appear at the top of the screen with a slide-down animation
• Auto-dismiss after 4 seconds with a fade-out animation
• Multiple notifications stack vertically
• Used across the entire website for user feedback:
  - 'Product added to cart!' (success)
  - 'Please log in first' (warning)
  - 'Error connecting to server' (error)
  - 'Custom bracelet saved!' (success)

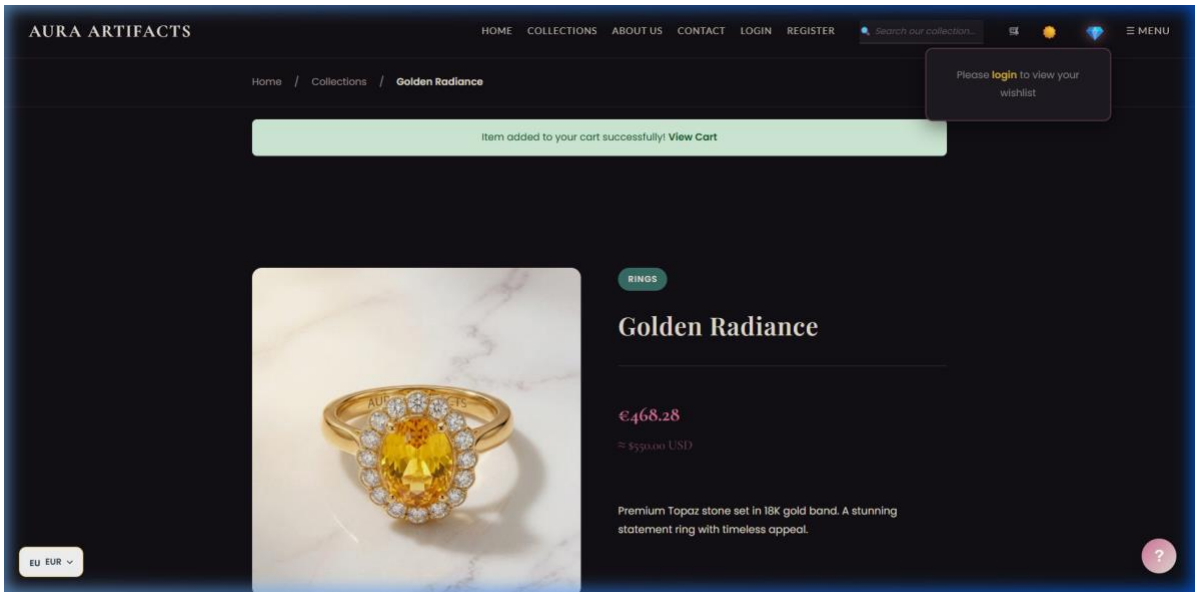


Figure: Toast Notification — 'Please login to view your wishlist' message appears when a guest user tries to add to wishlist. Also shows the success banner 'Item added to your cart successfully!'

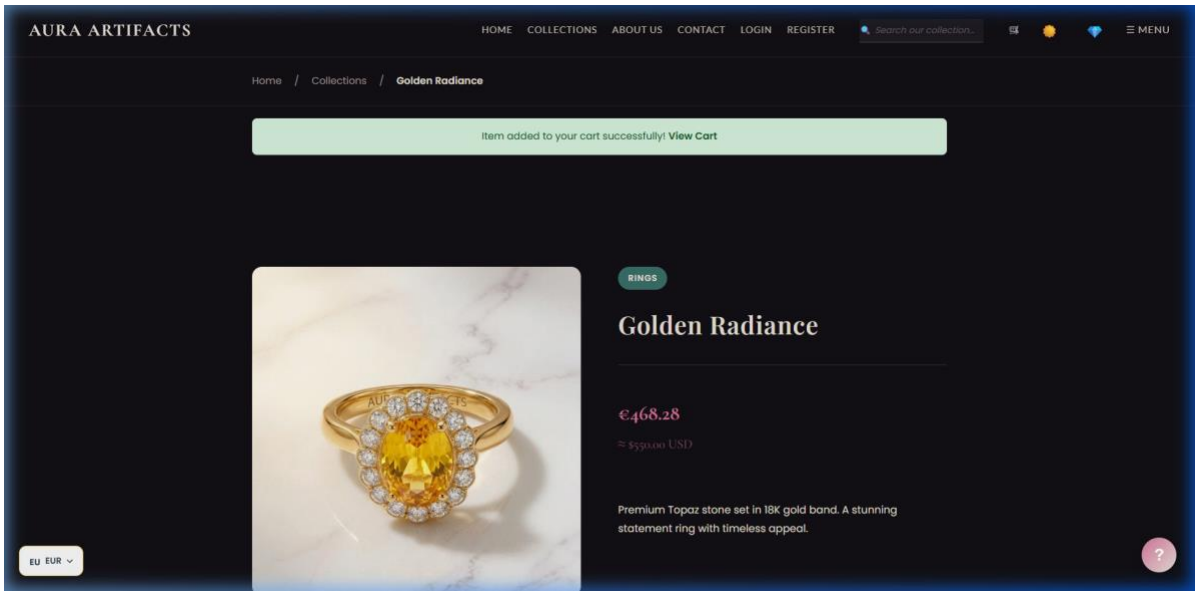


Figure: Toast Notification — 'Item added to your cart successfully! View Cart' success banner shown after adding a product to the shopping cart

Web Services & APIs Integrated into the System

The Aura Artifacts platform integrates two external web services/APIs:

API-1: Currency Exchange Rate API

Provider: ExchangeRate-API (open.er-api.com)
Type: RESTful Web Service
Endpoint: GET https://open.er-api.com/v6/latest/USD
Authentication: None (free public API)
Response Format: JSON
Integration File: assets/js/currency-converter.js

Purpose: Fetches live exchange rates for 150+ currencies to enable dynamic price conversion across the entire e-commerce platform. The API is called using the JavaScript fetch() function with async/await pattern. Rates are cached in localStorage for 1 hour.

Data Flow:
1. Browser sends GET request to open.er-api.com
2. API returns JSON with all exchange rates relative to USD
3. JavaScript parses the response and caches it
4. All price elements on the page are converted using the rates
5. User's currency preference is persisted in localStorage

API-2: Google Maps Embed API

Provider: Google (maps.google.com)
Type: Embedded Web Service (iframe)
Integration File: contact.php

Purpose: Displays an interactive Google Map showing the store's physical location (Al Jubail, Saudi Arabia) on the Contact Us page.

Implementation:
• An iframe embeds a Google Maps view directly into the page
• No API key required for basic embed functionality
• Map supports zooming, panning, and fullscreen
• Lazy loading enabled for performance (loading='lazy')
• Styled with border-radius and box-shadow to match website design

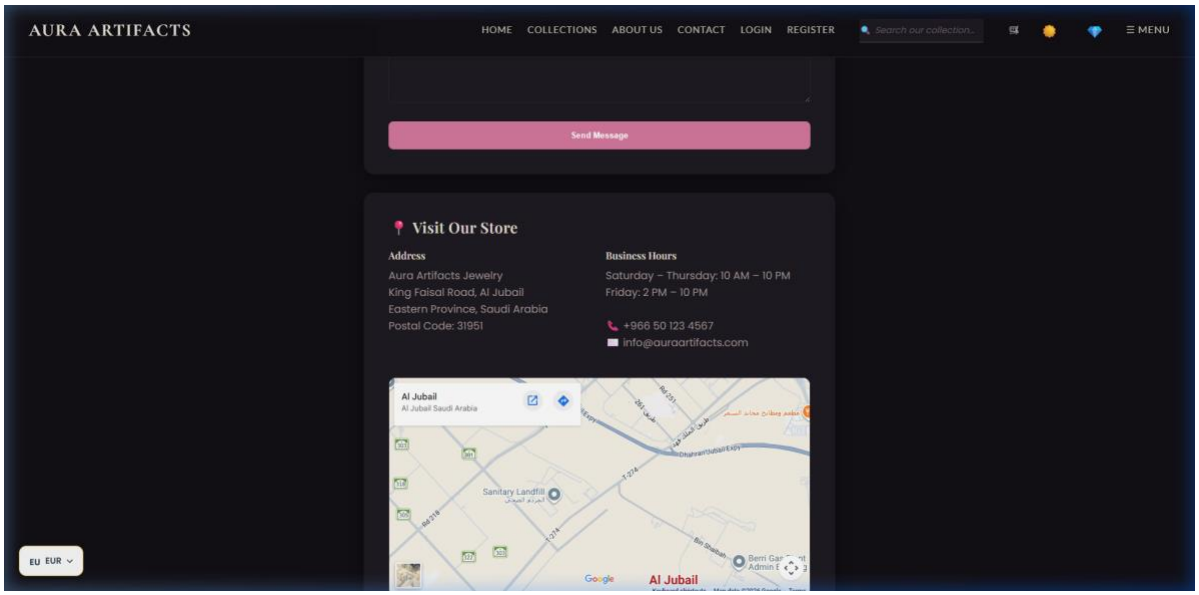


Figure: Google Maps Embed on the Contact Us page — showing the store location at Al Jubail, Saudi Arabia with address, business hours, phone number, and email. The interactive map supports zoom, pan, and fullscreen.